

CryptoChain Analyzer Dashboard

Project Report — Bitcoin Blockchain Analysis Dashboard

Student: Alex Trapero Lopez | GitHub: alextra05 | Course: Cryptography
Universidad Alfonso X el Sabio | Academic Year 2025–26

Section 1 — Cryptographic Metrics

1.1 Mining Difficulty and the SHA-256 Hash Space

Bitcoin's mining difficulty is a dimensionless measure of how hard it is to find a valid block hash. SHA-256 produces a 256-bit output, giving a hash space of $2^{256} \approx 1.16 \times 10^{77}$ possible values. A valid block hash must be numerically smaller than a 256-bit target threshold T . The difficulty D is defined relative to the genesis target T_{gen} (the easiest possible target, $0x00000000FFFF0000...$):

$$D = T_{\text{gen}} / T$$

As D increases, T decreases and miners must produce hashes with more leading zero bits, requiring exponentially more computational work. Bitcoin re-targets difficulty every 2016 blocks (≈ 2 weeks) to maintain an average inter-block time of 600 seconds, regardless of total network hash rate.

1.2 The bits Field — Compact Target Encoding

Rather than storing the full 256-bit target T in each block header (32 bytes), Bitcoin uses a 4-byte compact representation called **bits**, analogous to floating-point notation:

$$\begin{aligned} \text{bits} &= [\text{exponent}(1\text{ byte})] [\text{coefficient}(3\text{ bytes})] \\ T &= \text{coefficient} \times 256^{(\text{exponent} - 3)} \end{aligned}$$

The exponent encodes the byte length of T ; the coefficient provides the three most significant bytes of T . For example, `bits=0x1703a1c4` decodes to a target beginning with many leading zero bytes, reflecting the current high difficulty. The dashboard parses this field using Python's `struct.unpack` with little-endian byte order ('<I') and reconstructs T using the formula above.

1.3 Leading Zero Bits and Probabilistic Cost

A valid block hash must begin with at least k leading zero bits, where k is derived from T . Since each bit of a SHA-256 output is uniformly distributed, the probability of a single hash attempt satisfying the constraint is:

$$P(\text{valid}) \approx T / 2^{256} = 1 / 2^k$$

On average a miner must perform 2^k hash attempts to find one valid block. At the current difficulty ($k \approx 92$ leading zero bits) this equates to roughly 5×10^{27} hashes per block — achievable only through the collective work of hundreds of EH/s of network hash rate.

1.4 The 80-Byte Block Header Structure

Every Bitcoin block header is exactly 80 bytes, serialised in little-endian (LE) byte order as specified in the Bitcoin protocol. The six canonical fields are:

Field	Offset	Size	Description
version	0–3	4 B	Block version number (LE uint32). Signals soft-fork readiness.
prev_hash	4–35	32 B	SHA-256d hash of the previous block header (LE; reversed for display).
merkle_root	36–67	32 B	Root of the Merkle tree of all transactions in this block (LE).
timestamp	68–71	4 B	Unix epoch seconds (LE uint32). Set by miner; ± 2 h tolerance.
bits	72–75	4 B	Compact target threshold (LE uint32). Encodes 256-bit target T .
nonce	76–79	4 B	32-bit counter iterated by miner to find a valid hash (LE uint32).

Note: hash fields (prev_hash, merkle_root) are stored in LE and byte-reversed before display to match the big-endian convention used by block explorers.

1.5 SHA-256d — Double-Hash Proof-of-Work Verification

Bitcoin uses double SHA-256 (SHA-256d) to compute the block hash, providing additional protection against length-extension attacks. Verification in Python uses only the standard library:

```
hash = hashlib.sha256(hashlib.sha256(header_bytes).digest()).digest()
```

The dashboard fetches the raw 80-byte header hex from Blockstream's `/block/{hash}/header` endpoint, recomputes SHA-256d, byte-reverses the result, and compares it against the known block hash. A match confirms the header bytes are authentic. The result is then compared to `T` to verify the PoW constraint (`hash_int < T`).

1.6 Inter-block Times and the Exponential Distribution

Each hash attempt is an independent Bernoulli trial with success probability $p \approx 1/2^k$ per attempt. The entire network performs $\approx H$ hash attempts per second (where H is the network hash rate). This constitutes a **Poisson process** with rate $\lambda = H \times p$, and the inter-arrival times of a Poisson process follow an **Exponential distribution**:

$$X \sim \text{Exp}(\lambda), \quad E[X] = 1/\lambda \approx 600 \text{ s}$$

A key property of the Exponential distribution is **memorylessness**: $P(X > s+t \mid X > s) = P(X > t)$. The expected time to the next block is always ~ 10 minutes, regardless of how long miners have already been searching. This explains why consecutive blocks can be separated by just 30 seconds or by over an hour, even under stable conditions.

1.7 Network Hash Rate Estimation

The Bitcoin protocol does not expose the network hash rate directly. It is estimated from the current difficulty using the relation between expected hashes per block and target block time:

$$\text{hash_rate} \approx \text{difficulty} \times 2^{32} / 600$$

Derivation: the genesis difficulty-1 target requires on average 2^{32} hashes per block. A difficulty D therefore requires $D \times 2^{32}$ hashes per block on average. Dividing by the target block time (600 s) gives the estimated hash rate in H/s. At recent difficulty levels this equates to approximately 800–900 EH/s.

Section 2 — AI Component: Anomaly Detector

2.1 Model Chosen

Statistical Exponential Distribution Anomaly Detector. The M4 module implements a principled statistical detector based on the Exponential distribution, which is the theoretically correct model for Bitcoin inter-block times (see Section 1.6).

2.2 Why This Model

Inter-block times follow $\text{Exp}(\lambda = 1/600)$ by mathematical necessity — it is a consequence of the protocol design, not an empirical assumption. A statistical model tailored to this known distribution is therefore **more principled** than a black-box ML model (e.g., an Isolation Forest or autoencoder) trained on limited historical data, for the following reasons:

- **No training data required** — the distribution parameters are derived analytically from the Bitcoin protocol, not fitted to a sample.
- **Perfect generalisation** — the Exponential model holds for any block count and any network condition where the hash rate is approximately stationary.
- **Interpretable thresholds** — the anomaly cutoff (mean + 3σ) has a precise probabilistic meaning: under H , $P(X > \mu + 3\sigma) \approx e^{-9} \approx 5\%$, so approximately 5% of blocks are expected to exceed this threshold even without anomalies.
- **Adaptive** — the module uses the *empirical* mean and std from the fetched sample, so it remains accurate during periods of hash-rate transition.

2.3 Data Used

The detector fetches the timestamps of the **last 200 Bitcoin blocks** from the Blockstream public API (`GET /blocks/:height`) via the `get_recent_block_timestamps(200)` helper in `blockchain_client.py`. From these 200 timestamps, 199 inter-block time intervals are computed (sorted descending: $t[i] - t[i+1]$).

2.4 Anomaly Thresholds

Two independent thresholds flag a block as anomalous:

- **Upper tail (too slow):** $\text{time} > \mu_{\text{emp}} + 3\sigma_{\text{emp}}$ (≈ 2400 s / 40 min under typical conditions).
- **Lower floor (too fast):** $\text{time} < 30$ s — indicates possible miner timestamp manipulation or clock drift.

Both thresholds use the empirical mean (μ_{emp}) and standard deviation (σ_{emp}) computed from the current sample.

2.5 Evaluation Metrics

- **Total blocks analyzed:** number of inter-block intervals computed (target: 199).
- **Anomalies detected:** count of intervals exceeding either threshold.
- **Anomaly percentage:** $\text{anomalies} / \text{total} \times 100$ (expected $\sim 5\%$ under H_0).
- **Mean inter-block time:** empirical mean compared against the 600 s protocol target.

2.6 Limitations

- **Timestamp manipulation:** Bitcoin's BIP 0113 allows miners to set the block timestamp within a ± 2 -hour window relative to the network-adjusted time. Artificially small or large timestamps can produce spurious anomaly flags.
- **Non-stationary hash rate:** during events such as a large mining farm going offline (e.g., a power outage or regulatory shutdown) the inter-block time distribution temporarily departs from $\text{Exp}(1/600)$, producing elevated anomaly rates that are real events rather than individual block anomalies.
- **Sample size:** 200 blocks covers approximately 33 hours of Bitcoin history. The empirical mean may differ from 600 s by ± 20 – 30 s due to sampling variance even under perfectly normal conditions.

Section 3 — References

- [1] Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*. Available at: <https://bitcoin.org/bitcoin.pdf>
- [2] Blockstream. (2024). *Blockstream API Documentation*. Available at: <https://blockstream.info/api/>
- [3] NIST. (2015). *FIPS PUB 180-4: Secure Hash Standard (SHS)*. National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.FIPS.180-4>
- [4] Rosenfeld, M. (2012). *Analysis of Hashrate-Based Double Spending*. arXiv:1402.2009. Available at: <https://arxiv.org/abs/1402.2009>
- [5] Bitcoin Wiki. (2024). *Block hashing algorithm*. https://en.bitcoin.it/wiki/Block_hashing_algorithm
- [6] Bitcoin Wiki. (2024). *Difficulty*. <https://en.bitcoin.it/wiki/Difficulty>